

# Design Rationale R2: Golden Beetle and Interactions

## 1. Introduction

This document outlines the design rationale for "Requirement 2: Try Finger, But Hole," focusing on the introduction of the Golden Beetle, its unique Golden Egg, and associated interactions with the Farmer (Player). The design strictly adheres to the classes, interfaces, and relationships depicted in the provided UML diagram (REQ2.png) for this requirement. The goal is to integrate these new mechanics seamlessly, ensuring clarity, adherence to the UML, and consideration for future extensibility as outlined in the design considerations.

## 2. Design Goals

### 1. Implement Golden Beetle (GoldenBeetle) Creature:

- Define its attributes (25 HP, immune to Crimson Rot).
- Implement its item production capability (produces GoldenBeetleEgg every 5 turns) via the ItemProducible interface and ProduceBehaviour.
- Enable the Player to consume an adjacent GoldenBeetle directly (heal 15 HP, grant 1000 runes to Player's Wallet), making GoldenBeetle Eatable.
- Implement its wandering and following movement using FollowBehaviour, with egg production taking priority.
- Allow the Player to attack the GoldenBeetle using AttackAction.

### 2. Implement Golden Egg (GoldenBeetleEgg) Mechanics:

- Define it as an Egg that is Eatable.
- Implement its unique hatching condition: hatches into a new GoldenBeetle only when near "cursed entities" (e.g., Blight).
- Ensure it does not hatch if in the Player's inventory.
- Allow the Player to consume a GoldenBeetleEgg from inventory to restore 20 stamina.

### 3. Integrate with Existing Systems:

- Utilize the Eatable interface and EatAction for consuming both the GoldenBeetle and GoldenBeetleEgg.
- Leverage the ItemProducible interface for egg production.
- Incorporate the Player's Wallet for rune management.

### 3. Key Design Components & Interactions

#### a) Wallet

##### Role & Responsibilities:

- Associated with the Player actor.
- Manages the Player's currency (runes).
- Provides methods to add or deduct runes.

##### OOP Principles & Justification:

- **Single Responsibility Principle (SRP):** Its sole responsibility is to manage runes.
- **Encapsulation:** Hides the internal representation of the rune balance.

| Pros                                                      | Cons                                                                                                                  |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Centralizes currency management for the Player.           | Relies on a turn management system to trigger its production logic at the correct interval.                           |
| Decouples rune handling from the main Player class logic. | Other game systems or actions (like consuming GoldenBeetle or purchasing items) need to interact with this component. |

#### b) GoldenBeetle

##### Role & Responsibilities:

- Represents the Golden Beetle. Implements ItemProducible (via ProduceBehaviour) and Eatable.
- Manages its health (25 HP), immunity to rot, and its specific behaviours like producing GoldenBeetleEggs, wandering, following (via FollowBehaviour), and being consumable by the Player.

##### OOP Principles & Justification:

- **Encapsulation:** Contains its state (health, target to follow) and specific logic for its unique behaviors.
- **Implementation of Interfaces (ItemProducible, Eatable):** Fulfills the contracts for producing items and being consumable.
- **Delegation (to Behaviours):** Uses FollowBehaviour and ProduceBehaviour

to implement parts of its AI and item production logic, promoting SRP for the behaviours.

| Pros                                                                                                                                                 | Cons                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Clearly defines the Golden Beetle's multifaceted nature.                                                                                             | Coordination between its own state, its behaviours, and game systems (like turn management for production) needs careful implementation. |
| The use of behaviour classes (FollowBehaviour, ProduceBehaviour) makes its AI and production logic modular and potentially reusable or configurable. | The "consumable directly from map" aspect requires EatAction to handle targets not in inventory, or a specialized action.                |

### c) GoldenBeetleEgg

#### Role & Responsibilities:

- Concrete Egg subclass, implements Eatable.
- Encapsulates its specific hatching logic (near "cursed entities") and its edibility effect (restores 20 stamina).

#### OOP Principles & Justification:

- **Inheritance & Specialization.**
- **SRP (for its specific concerns):** Manages its unique hatching and eating outcomes.

| Pros                                                                   | Cons                                                                                                                                                                 |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clearly defines the properties and behaviors of a Golden Beetle's egg. | The "near cursed entities" check for hatching requires interaction with the game map/world to identify surroundings and their properties. This logic must be robust. |

|                                                                                           |  |
|-------------------------------------------------------------------------------------------|--|
| Allows for unique, environmentally-triggered hatching conditions, showcasing flexibility. |  |
|-------------------------------------------------------------------------------------------|--|

#### d) Player

**Role & Responsibilities:** (Similar to REQ1, but with added Wallet interaction and stamina effects.

- Manages Wallet for runes.
- Initiates EatAction on GoldenBeetle (direct) and GoldenBeetleEgg (inventory).
- Initiates AttackAction on GoldenBeetle.

#### e) AttackAction (from Assignment 1)

**Role & Responsibilities:**

- Represents an attack performed by an actor (e.g., Player) on a target creature (e.g., GoldenBeetle).
- Handles the logic of applying damage or other attack effects

#### f) FollowBehaviour

**Role & Responsibilities:**

- Encapsulates the AI logic for an actor (GoldenBeetle) to follow another target actor (Player).
- Determines movement towards the target. Manages state (e.g., current target).

**OOP Principles & Justification:**

- **Strategy Pattern / Behavioural Pattern, SRP:** Isolates a specific AI behavior.
- **Encapsulation:** Hides the details of how following is implemented.

| Pros                                                                        | Cons                                                                |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------|
| Makes AI logic modular and reusable for other creatures or other followable | Requires access to the game map for pathfinding/movement and to the |

|                                                                |                                                                     |
|----------------------------------------------------------------|---------------------------------------------------------------------|
| targets.                                                       | actor's state.                                                      |
| Simplifies the GoldenBeetle class by delegating this behavior. | Needs a mechanism to detect followable targets in the surroundings. |

## g) ProduceBehaviour

### Role & Responsibilities:

- Encapsulates the logic for an actor (GoldenBeetle) to produce an item (GoldenBeetleEgg) based on certain conditions (e.g., every 5 turns).
- This behaviour likely implements or is used by the ItemProducible interface methods in GoldenBeetle.

### OOP Principles & Justification:

- **Strategy Pattern / Behavioural Pattern, SRP:** Isolates item production logic.

| Pros                                                                      | Cons                                                                        |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Modularizes item production logic.                                        | Relies on a turn system or other triggers to activate its production check. |
| Can be configured for different items, timings, or production conditions. | Needs to interact with the game map to place the produced item.             |